School of Computer Science and Artificial Intelligence

Lab Assignment # 15.5

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

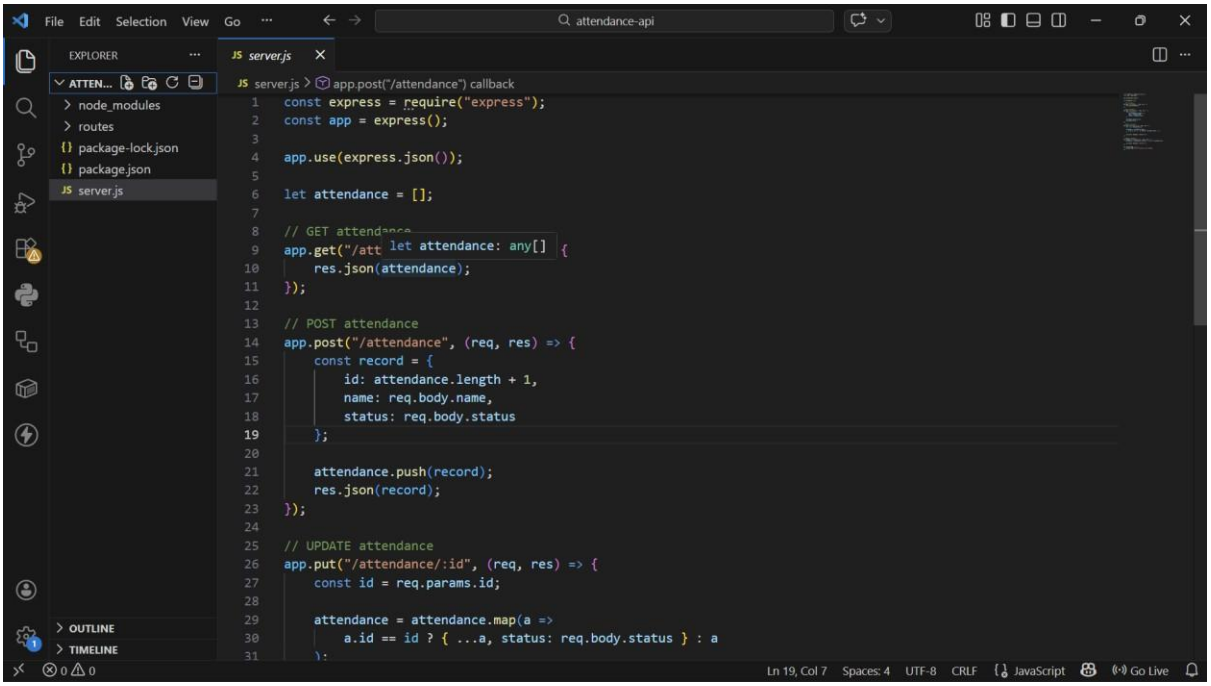
Name of Student : Swetha Dixit

Enrollment No. : 2303A52415

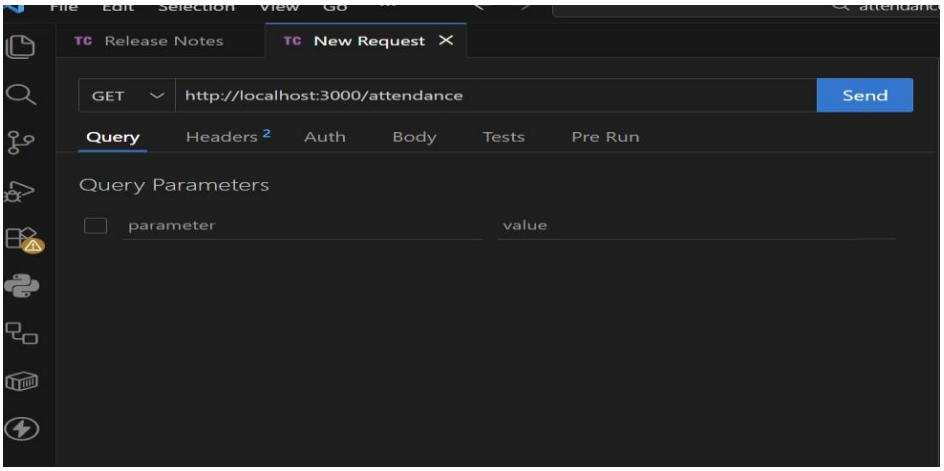
Batch No. : 33

Date :13/03/26

Task 1 – Student Attendance API



GET /attendance → View attendance records



o POST /attendance → Mark attendance

POST http://localhost:3000/attendance Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQ

```
1 {
2 "name": "Sowmya",
3 "status": "Present"
4 }
```

Status: 200 OK Size: 43 Bytes Time: 9 ms Response

```
1 {
2   "id": 1,
3   "name": "Sowmya",
4   "status": "Present"
5 }
```

o PUT /attendance/{id} → Update attendance record

TC Release Notes TC New Request

PUT http://localhost:3000/attendance/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

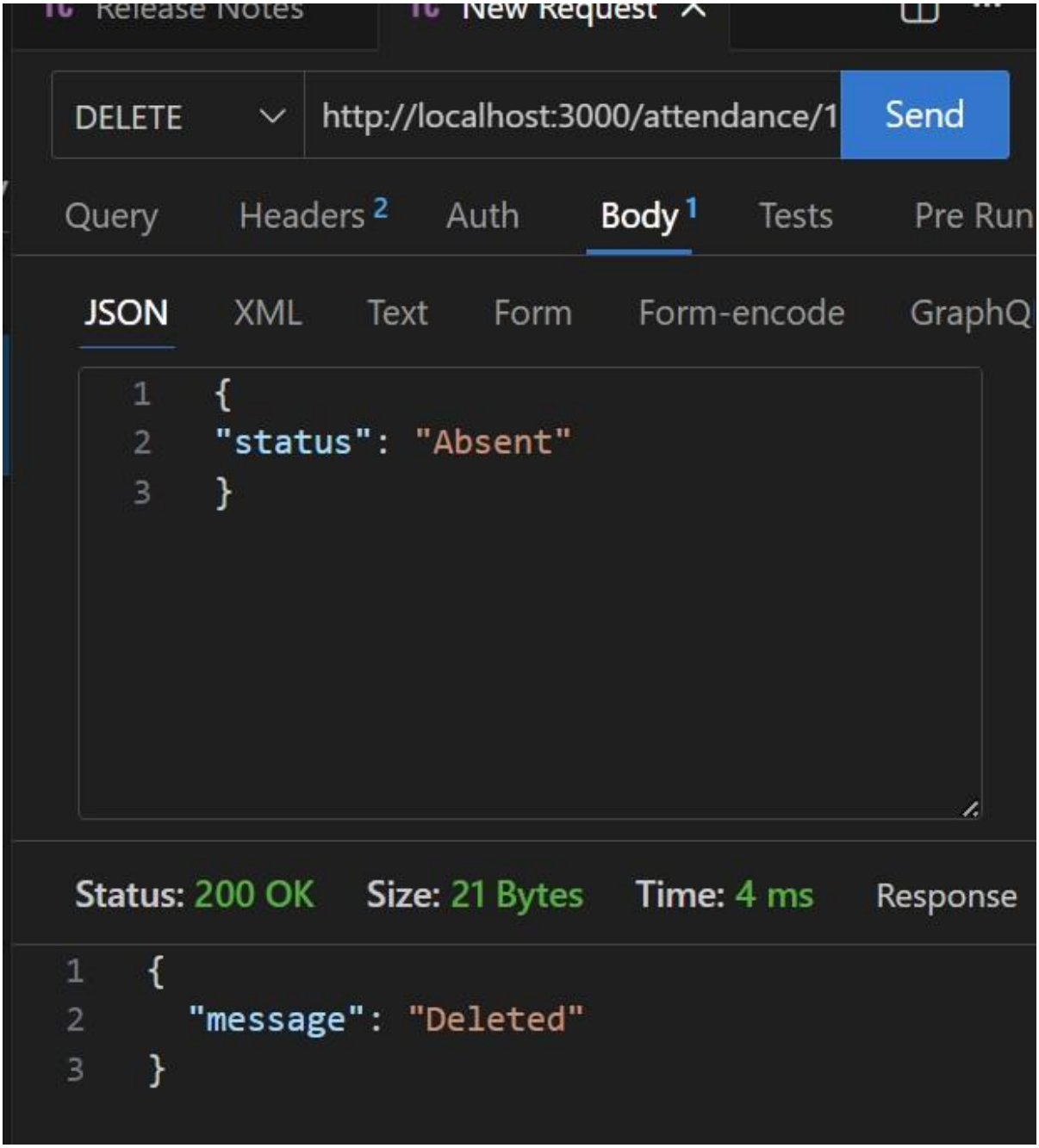
JSON XML Text Form Form-encode GraphQ

```
1 {
2 "status": "Absent"
3 }
```

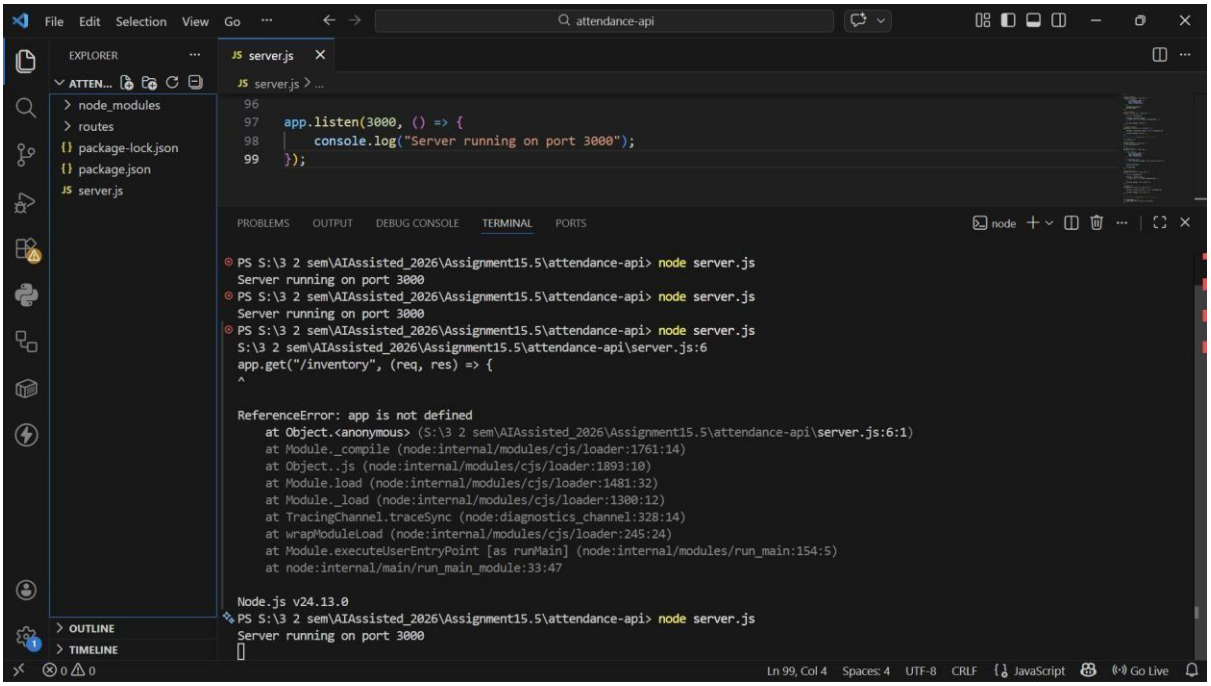
Status: 200 OK Size: 21 Bytes Time: 4 ms Response

```
1 {
2   "message": "Updated"
3 }
```

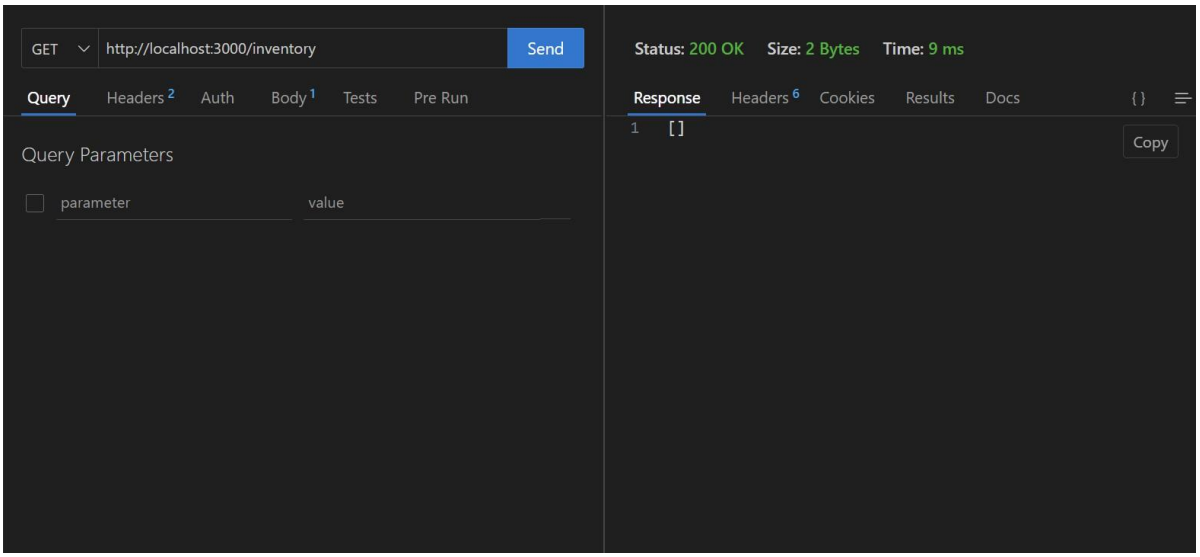
o DELETE /attendance/{id} → Remove attendance entry



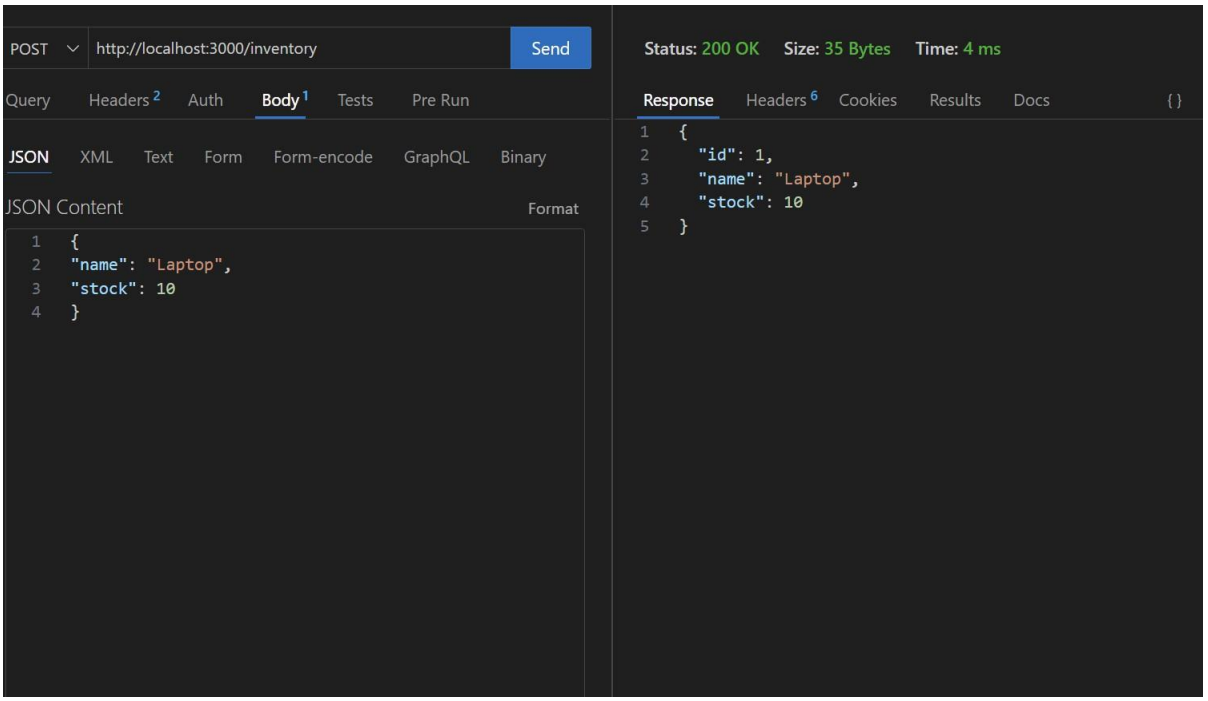
Task 2 – Inventory Management API



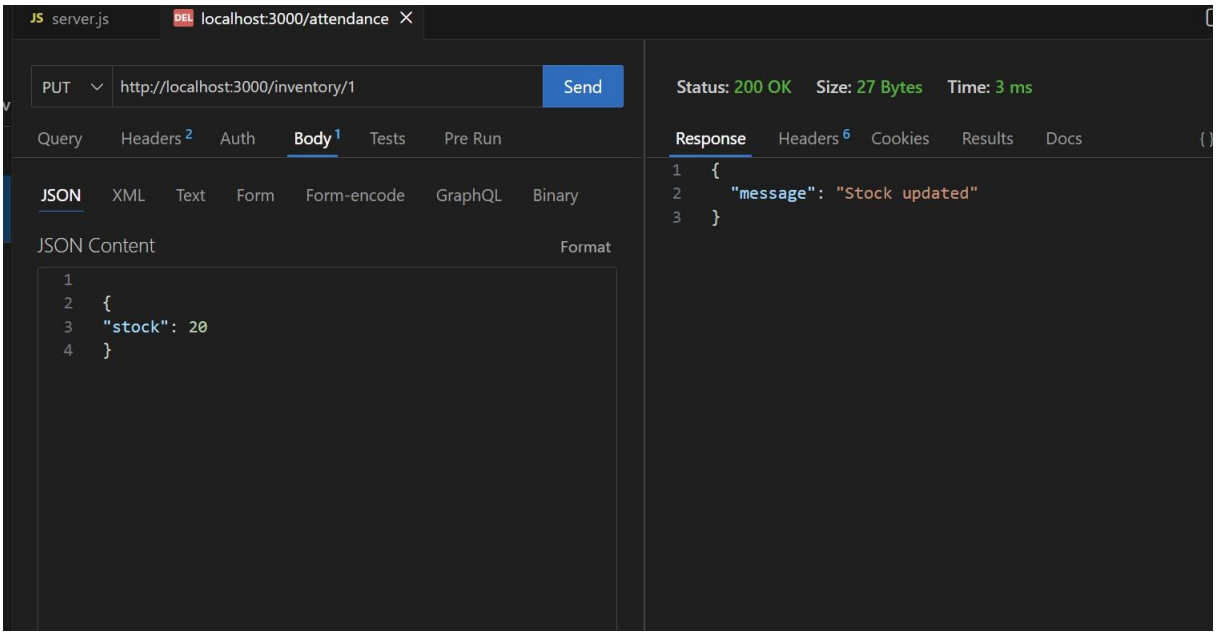
GET /inventory → List all items



o POST /inventory → Add new item



o PUT /inventory/{id} → Update stock quantity



o DELETE /inventory/{id} → Remove an item

server.js localhost:3000/attendance

DELETE http://localhost:3000/inventory/1 Send

Status: 200 OK Size: 26 Bytes Time: 5 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2 "message": "Item removed"
3 }
```

Response Headers 6 Cookies Results Docs

```
1 {
2   "message": "Item removed"
3 }
```

Task 3 – Hotel Room Booking API

server.js

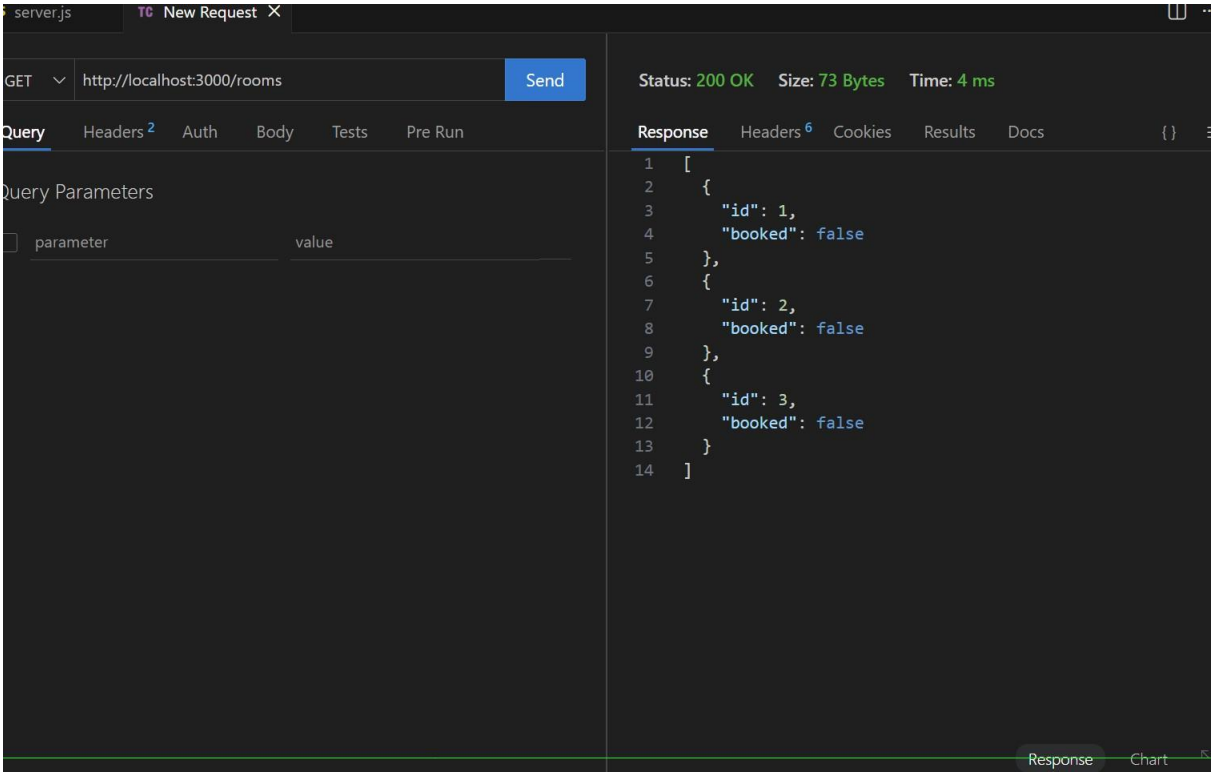
server.js > ...

```
151 app.delete("/rooms/:id", (req, res) => {
153     const room = rooms.find(r => r.id == req.params.id);
154
155     if (!room) {
156         return res.json({ message: "Room not found" });
157     }
```

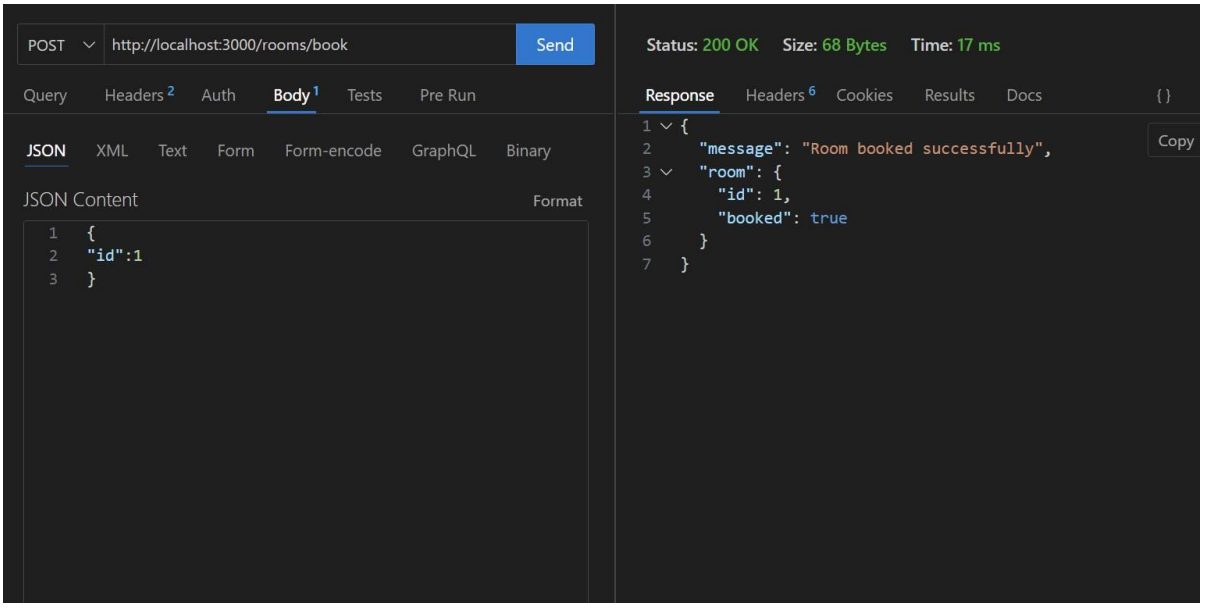
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS node

```
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6
app.get("/inventory", (req, res) => {
^

ReferenceError: app is not defined
    at Object.<anonymous> (S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6:1)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.13.0
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
```

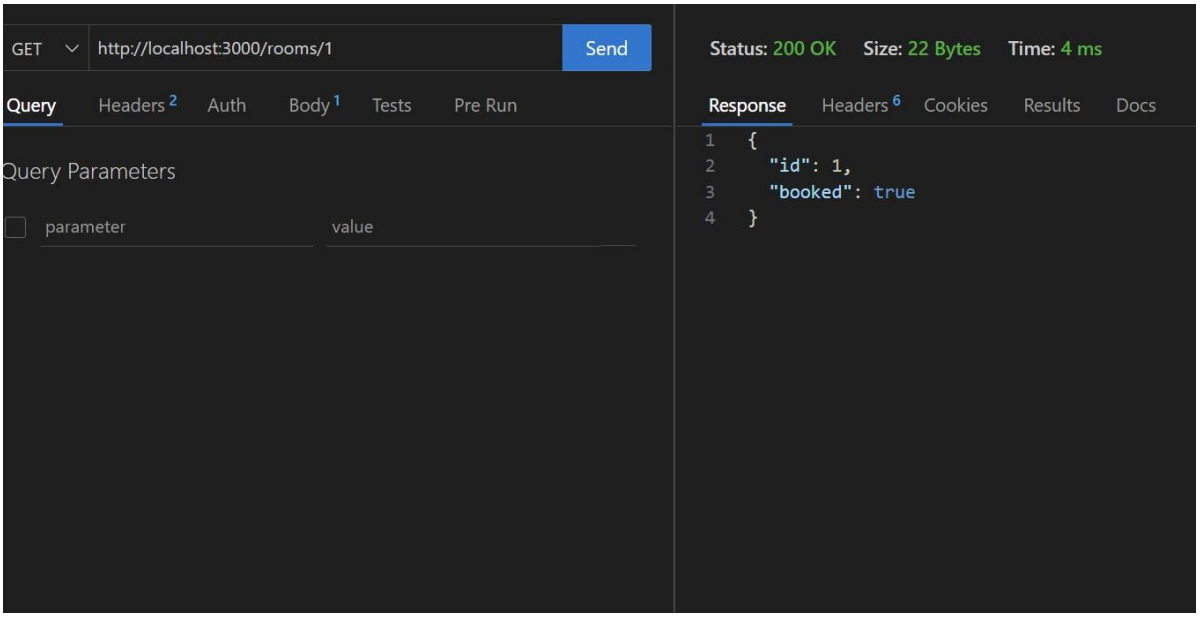
o GET /rooms → View available rooms



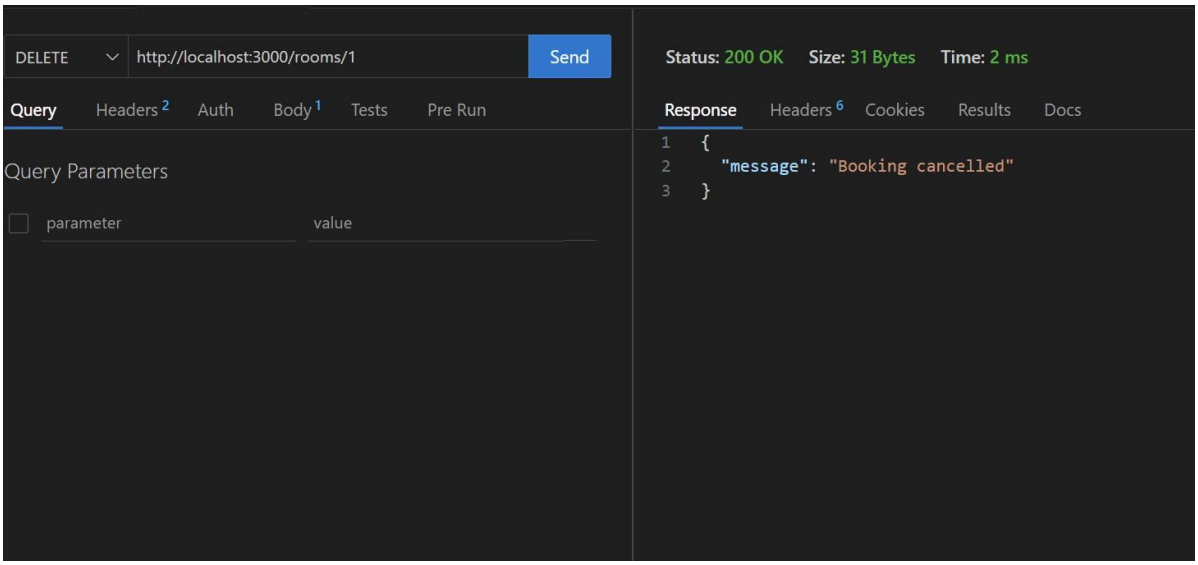
o POST /rooms/book → Book a room



o GET /rooms/{id} → View booking details



o DELETE /rooms/{id} → Cancel booking



Task 4 – Online Voting API

```javascript
// Import express
const express = require("express");
const app = express();

app.use(express.json());

// Candidate data
let candidates = [
    { id: 1, name: "Alice", votes: 0 },
    { id: 2, name: "Bob", votes: 0 },
    { id: 3, name: "Charlie", votes: 0 }
];

// GET all candidates
app.get("/candidates", (req, res) => {
    res.json(candidates);
});

// POST vote for candidate
app.post("/vote", (req, res) => {

    const candidate = candidates.find(c => c.id == req.body.id);

    if (!candidate) {
        return res.json({ message: "Candidate not found" });
    }

    candidate.votes++;

    res.json({
        message: "Vote recorded"
```

o GET /candidates → List candidates

GET http://localhost:3000/candidates Send

Status: 200 OK Size: 103 Bytes Time: 8 ms

```json
[
  {
    "id": 1,
    "name": "Alice",
    "votes": 0
  },
  {
    "id": 2,
    "name": "Bob",
    "votes": 0
  },
  {
    "id": 3,
    "name": "Charlie",
    "votes": 0
  }
]
```

o POST /vote → Cast a vote

POST http://localhost:3000/vote Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
"id":1
}
```

Status: 200 OK Size: 73 Bytes Time: 9 ms

Response Headers 6 Cookies Results Docs

```
{
  "message": "Vote recorded",
  "candidate": {
    "id": 1,
    "name": "Alice",
    "votes": 1
  }
}
```

o GET /results → View voting results

GET http://localhost:3000/results Send

Query Headers 2 Auth Body 1 Tests Pre Run

Query Parameters

parameter	value

Status: 200 OK Size: 158 Bytes Time: 3 ms

Response Headers 6 Cookies Results Docs { }

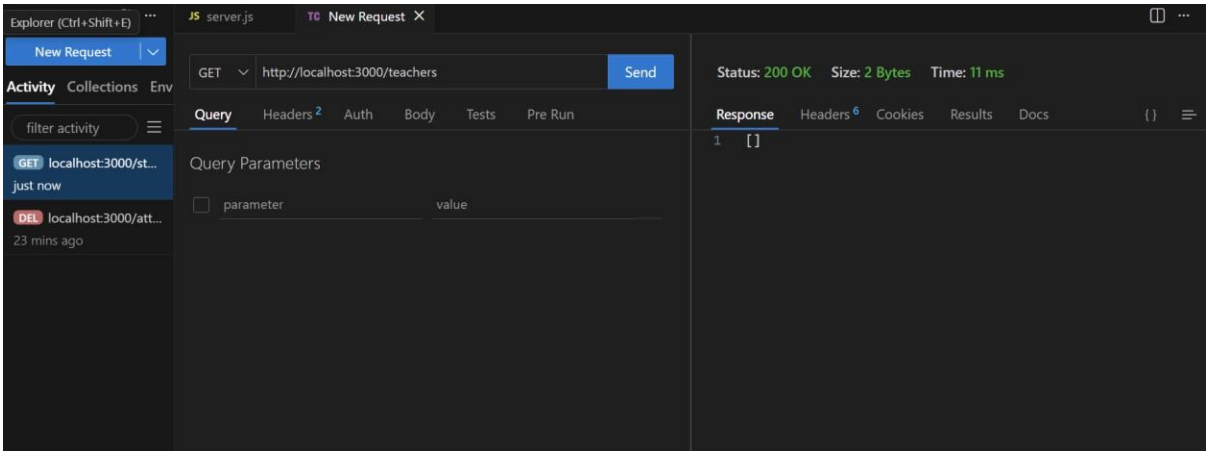
Copy

```
{
  "results": [
    {
      "id": 1,
      "name": "Alice",
      "votes": 1
    },
    {
      "id": 2,
      "name": "Bob",
      "votes": 0
    },
    {
      "id": 3,
      "name": "Charlie",
      "votes": 0
    }
  ],
  "winner": {
    "id": 1,
    "name": "Alice",
    "votes": 1
  }
}
```

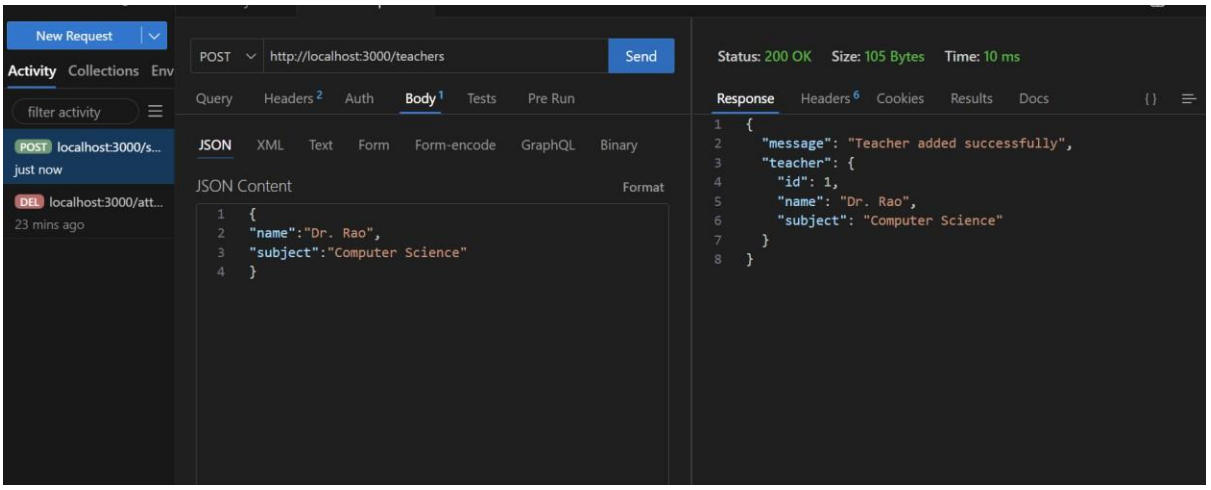
Task 5 – Teacher Management API

```
JS server.js  ×   TC New Request
JS server.js > app.put("/teachers/:id") callback > message
37  app.put("/teachers/:id", (req, res) => {
45      teacher.name = req.body.name || teacher.name;
46      teacher.subject = req.body.subject || teacher.subject;
47
48      res.json({
49          message: "Teacher updated",
50          teacher: teacher
51      });
52  });
53
54  // DELETE /teachers/:id → Remove teacher
55  app.delete("/teachers/:id", (req, res) => {
56
57      const teacher = teachers.find(t => t.id == req.params.id);
58
59      if (!teacher) {
60          return res.status(404).json({ message: "Teacher not found" });
61      }
62
63      teachers = teachers.filter(t => t.id != req.params.id);
64
65      res.json({ message: "Teacher removed successfully" });
66  });
67
68  // Start server
69  app.listen(3000, () => {
70      console.log("Server running on port 3000");
71  });
```

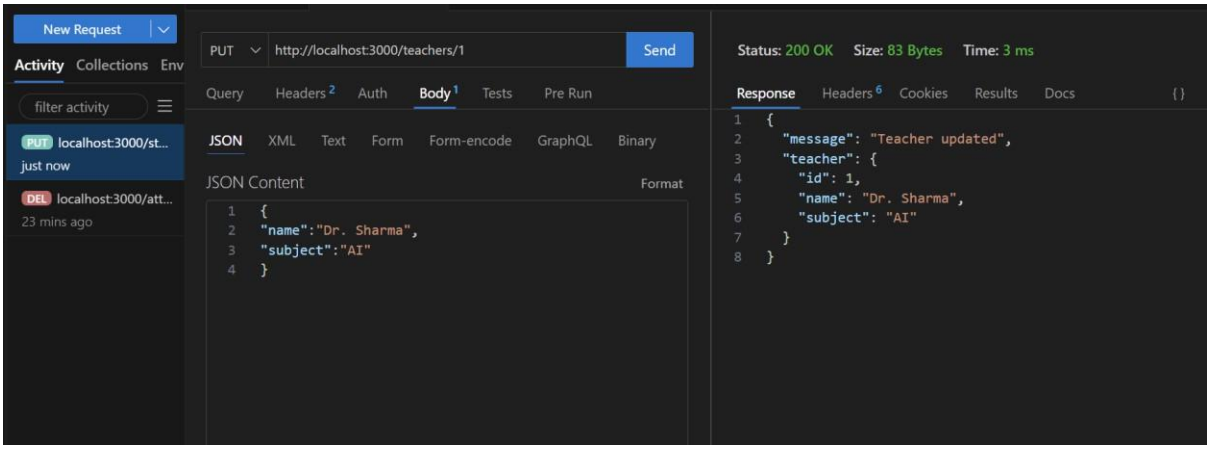
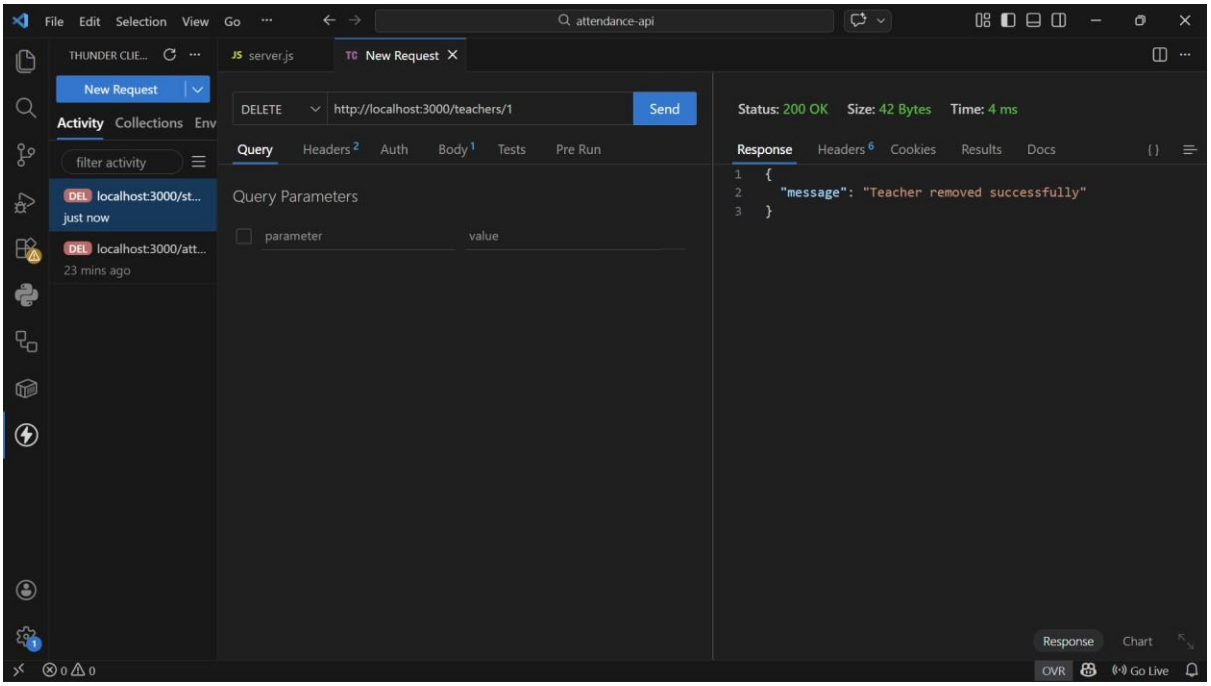
o GET /teachers → List all teachers



o POST /teachers → Add a new teacher



o PUT /teachers/{id} → Update teacher details



o DELETE /teachers/{id} → Remove a teacher